B8 Statistical correctness

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B8 Statistical correctness 3
B8-L1 Survey weights and the effective sample size 3
B8-L2 Confidence intervals and effect sizes 7
B8-L3 Many tests, and the false positives they bring 11
Track B best-practice checklist 16
Appendix: solutions and answer keys 17

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B8 Statistical correctness

Roadmap: Prerequisite for all months. Survey weights, confidence intervals, effect sizes and multiple comparisons.

B8-L1 Survey weights and the effective sample size

Concept

A survey sample rarely matches its population. If young adults are 20% of the population but 10% of your completes, each young respondent gets a larger **weight** so the weighted sample matches the population.

The **weighted mean** divides by the sum of the weights, not by the row count:

```
weighted_mean = (y * w).sum() / w.sum()      # same as np.average(y, weights=w)
```

Weights have a cost. Unequal weights make estimates less precise than the raw sample size suggests. Kish's **effective sample size** measures that cost:

```
n_eff = w.sum() ** 2 / (w ** 2).sum()
design_effect = n / n_eff
```

Equal weights give `n_eff = n`. A sample of 5 with weights 0.5, 0.5, 1, 2 and 4 has `n_eff = 2.98`: it carries about as much information as three unweighted respondents. The design effect is 1.68.

Rules that prevent the common mistakes:

- **Use weights for population estimates:** shares, means and totals you report as "UK adults".
- **Use `n_eff`, not `n`, for precision:** confidence intervals, significance tests and minimum base rules (lesson B8-L2).
- **Report the unweighted base** next to every weighted figure, so readers see how many people stand behind it.
- **Drop rows where the value or the weight is missing** before you compute. A missing weight is not zero, and a missing answer is not a zero answer.
- **Guard the division.** An empty subgroup or a zero weight total has no mean. Return `None`.

Check weights before using them: all positive, none missing, and a sum equal to the sample size or the population total, whichever the weighting

scheme promises. Extreme weights (above about 5 times the mean) are a warning sign. Trimming them is a design decision, logged as an assumption (lesson B7-L3).

Why it matters

A brand tracker reported that awareness among 18 to 24s rose from 41% to 52%, "significant at 95%". The test used the 180 raw respondents in the group. Their weights were uneven, and the effective sample size was 71. With 71, the change was well inside the margin of error. The client moved budget on a result the data did not support.

Worked example: weak code

```
share = (df["aware"] * df["weight"]).mean()
se = (share * (1 - share) / len(df)) ** 0.5
```

1. **Line 1.** Divides by the row count, not the weight total. Only right if weights average exactly 1.
2. **Line 2.** Uses raw n, overstating precision whenever weights vary.

Worked example: strong code

```
import pandas as pd


def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float | None:
    """Weighted mean of value, ignoring rows with a missing value or weight."""
    d = df[[value, weight]].dropna()
    total = d[weight].sum()
    if len(d) == 0 or total == 0:
        return None
    return float((d[value] * d[weight]).sum() / total)


def effective_n(weights: pd.Series) -> float | None:
    """Kish effective sample size: (sum w)^2 / sum(w^2)."""
    w = weights.dropna()
    sum_sq = (w**2).sum()
    if sum_sq == 0:
        return None
    return float(w.sum() ** 2 / sum_sq)
```

1. **Line 6.** Rows missing either the answer or the weight do not count.
2. **Line 10.** Divides by the weight total.
3. **Line 19.** The n to use for precision.

Weighted estimates divide by the weight total. Precision uses the effective sample size.

Recap card

- **Weighted mean** sum(y * w) / sum(w), over rows with both present. Same as np.average(y, weights=w).
- **Kish effective sample size** (sum w)^2 / sum(w^2). Design effect = n / n_eff.
- **Which n for intervals and suppression?** n_eff. Report the raw base as well.
- **Missing weight** Not zero. Drop the row from weighted estimates, and investigate.

Practice

Exercise 1. Multiple choice (level 1)

A subgroup has 180 respondents and an effective sample size of 71. Which number belongs in the standard error of its weighted share?

- a) 180
- b) 71
- c) The sum of the weights
- d) 180 minus 71

Why?

- r1) Unequal weights reduce precision, and n_eff measures the information that remains.
- r2) Smaller numbers are always safer.

Exercise 2. Predict the output (level 2)

What does this print? (NumPy 2.4)

```
import numpy as np

y = np.array([1, 0, 1, 0, 1])
w = np.array([0.5, 0.5, 1.0, 2.0, 4.0])
print(np.average(y, weights=w), y.mean())
```

Exercise 3. Fill the gap (level 2)

Compute Kish's effective sample size and the design effect.

```
n_eff = w.sum() ** 2 / (w ** _____(1)).sum()
deff = _____(2) / n_eff
```

Exercise 4. Write the function (level 3)

Write two functions.

- `weighted_share(df, value, weight) -> float | None`: the weighted mean of `value`, ignoring rows where the value or the weight is missing. Return `None` when no rows remain or the weight total is 0.
- `effective_n(weights: pd.Series) -> float | None`: Kish's effective sample size, ignoring missing weights. Return `None` when the sum of squared weights is 0.

Your code must pass these checks:

- Divide sum(value * weight) by sum(weight), over rows with both present.
- An empty frame or a zero weight total must return None.
- n_eff = sum(w)^2 / sum(w^2), ignoring missing weights. Equal weights give n.
- No weights must return None, not ZeroDivisionError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Weighted mean of value over rows with both value and weight present.
def weighted_share(
    df: pd.DataFrame,
    value: str,
    weight: str,
) -> float | None: ...

# Kish effective sample size, (sum w)^2 / sum(w^2), ignoring missing
  weights.
def effective_n(weights: pd.Series) -> float | None: ...
```

Starter code:

```python
import pandas as pd


def weighted_share(df, value, weight):
    return (df[value] * df[weight]).mean()


def effective_n(weights):
    return len(weights)
```

Exercise 5. Architecture design (level 4)

Choose the right quantity for each part of a weighted subgroup report.

Options: Weighted estimate, Unweighted estimate, Raw base (n), Effective sample size.

1. The share shown in the chart
2. The n in the confidence interval
3. The base printed under the chart
4. The number compared with the minimum reporting base

Sources

- Kish effective sample size is (sum w)^2 / sum(w^2). (verified: Kish (1965), Survey Sampling; computed with NumPy 2.4.6 in this repository.)

B8-L2 Confidence intervals and effect sizes

Concept

A share without an interval is a guess with no stated size. Report every estimate with a **confidence interval**, and every difference with an **effect size**.

Intervals for a proportion. The textbook Wald interval, `p ± 1.96 * sqrt(p(1-p)/n)`, fails at the edges. For 0 out of 20 it gives `[0, 0]`, as if you were certain. The **Wilson** interval behaves well for small `n` and for shares near 0 or 1. For 0 out of 20 it gives `[0, 0.161]`. In statsmodels 0.14:

```
from statsmodels.stats.proportion import proportion_confint
low, high = proportion_confint(count=40, nobs=100, alpha=0.05,
    method="wilson")   # (0.309, 0.498)
```

For a **weighted** share, use the weighted estimate and the effective sample size from lesson B8-L1 in the same formula. The 18 to 24 group from that lesson, with a weighted share of 52% and `n_eff = 71`, has a Wilson interval of 40.6% to 63.2%. With the raw 180 it would be 44.7% to 59.2%, which looks far more precise than the data is.

Effect sizes. A p-value answers "is there any difference?", and with a large sample the answer is nearly always yes. Two groups of 100,000 with shares of 50.5% and 50.0% give p = 0.025, and the difference is too small to matter. State the size:

- The **difference in shares**, in percentage points, with its interval. This is what a client reads.

- **Cohen's h**, `2*asin(sqrt(p1)) - 2*asin(sqrt(p2))`, compares differences across different base rates. Rough guides: 0.2 small, 0.5 medium, 0.8 large.
- For means, the difference with its interval, or **Cohen's d** (difference over pooled standard deviation).

Decide before the analysis what size of difference matters for the decision. A significant difference below that size is reported as "no meaningful change".

Why it matters

A retailer's panel of 90,000 customers showed satisfaction at 7.62 against 7.58 the previous quarter, p = 0.01. The headline read "Satisfaction significantly up". The store managers' bonus scheme was tied to "significant improvement", and bonuses were paid. The change was 0.04 points on a 10-point scale, smaller than the rounding in the published chart.

Worked example: weak code

```
se = (p * (1 - p) / n) ** 0.5
low, high = p - 1.96 * se, p + 1.96 * se
if p_value < 0.05:
    headline = "Significant increase"
```

1. **Line 1.** Wald: zero width at p = 0 or 1, and too narrow for small n.
2. **Line 4.** Significance without size: true for trivial differences in large samples.

Worked example: strong code

```
import math


def wilson_interval(p: float, n: float, z: float = 1.959964) ->
  tuple[float, float] | None:
    """Wilson interval for a share p from n observations (use n_eff when
  weighted)."""
    if n <= 0 or not 0 <= p <= 1:
        return None
    denom = 1 + z**2 / n
    centre = (p + z**2 / (2 * n)) / denom
    half = z / denom * math.sqrt(p * (1 - p) / n + z**2 / (4 * n**2))
    return (max(0.0, centre - half), min(1.0, centre + half))


def cohens_h(p1: float, p2: float) -> float:
    """Effect size for two shares, comparable across base rates."""
    return 2 * math.asin(math.sqrt(p1)) - 2 * math.asin(math.sqrt(p2))
```

1. **Line 4.** n is a float, so n_eff from weights fits.

2. **Line 6.** No interval for an empty group or an impossible share.
3. **Line 16.** An effect size to report beside the difference in points.

Wilson intervals with n_eff for weighted shares, and an effect size beside every p-value.

Recap card

- **Why not the Wald interval?** Zero width at p = 0 or 1, and too narrow for small n. Use Wilson.
- **Interval for a weighted share** Wilson with the weighted p and n_eff.
- **Significant vs meaningful** Large samples make trivial differences significant. Report the size with its interval.
- **Cohen's h** 2asin(sqrt(p1)) - 2asin(sqrt(p2)). About 0.2 small, 0.5 medium, 0.8 large.

Practice

Exercise 1. Multiple choice (level 1)

0 of 20 respondents chose a new brand. What does the Wald interval `p ± 1.96 * sqrt(p(1-p)/n)` give?

- a) [0, 0]
- b) [0, 0.161]
- c) [-0.05, 0.05]
- d) It raises an error.

Why?

- r1) With p = 0 the standard error is 0, so the interval collapses.
- r2) Twenty respondents is too few for any interval.

Exercise 2. Predict the output (level 2)

What does this print? (statsmodels 0.14)

What you need to know.

`proportions_ztest(count, nobs)` from `statsmodels.stats.proportion` tests whether two proportions differ. `count` holds the successes in each group, and `nobs` the group sizes. It returns the z statistic and the two-sided p-value.

```python
from statsmodels.stats.proportion import proportions_ztest

z, p = proportions_ztest([30, 45], [100, 100])   # 30% against 45%
# z = -2.19, p = 0.028
```

```python
from statsmodels.stats.proportion import proportions_ztest
```

```
2
3 stat, p_value = proportions_ztest([50500, 50000], [100000, 100000])
4 print(p_value < 0.05)
```

Exercise 3. Fill the gap (level 2)

Get a 95% Wilson interval for 40 out of 100 with statsmodels.

```
1 low, high = proportion_confint(count=40, nobs=100, alpha=_____(1),
      method="_____(2)")
```

Exercise 4. Write the function (level 3)

Write `wilson_interval(p: float, n: float, z: float = 1.959964) -> tuple[float, float] | None` with the formula:

- `denom = 1 + z**2 / n`
- `centre = (p + z**2 / (2 * n)) / denom`
- `half = z / denom * sqrt(p * (1 - p) / n + z**2 / (4 * n**2))`

Return `(centre - half, centre + half)`, clipped to [0, 1]. Return `None` when `n <= 0` or `p` is outside [0, 1]. `n` is a float so you pass `n_eff` for weighted shares.

Your code must pass these checks:

- 0.4 from 100 must give (0.3094, 0.4980). Check the Wilson formula.
- 0 from 20 must give (0, 0.1611), not a zero-width interval.
- A float n_eff such as 71.0 must work.
- Return None for n <= 0 or p outside [0, 1].

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
1 # Wilson score interval for a share.
2 def wilson_interval(
3     p: float,
4     n: float,
5     z: float = 1.959964,
6 ) -> tuple[float, float] | None: ...
```

Starter code:

```
1 import math
2
3
4 def wilson_interval(p, n, z=1.96):
5     se = math.sqrt(p * (1 - p) / n)
6     return (p - z * se, p + z * se)
```

Exercise 5. Architecture design (level 4)

A panel of 90,000 shows satisfaction of 7.62 against 7.58 last quarter, p = 0.01. The team agreed beforehand that 0.2 points matters for decisions. Choose what to report.

Options: No meaningful change in satisfaction, Satisfaction significantly up.

1. Headline
2. Supporting detail
3. Rule for future quarters

Sources

- statsmodels proportion_confint(40, 100, method="wilson") returns (0.3094, 0.4980); method="normal" gives (0, 0) for 0 of 20. (verified: Run on statsmodels 0.14.6 and 0.15.0 in this repository.)
- proportions_ztest([50500, 50000], [100000, 100000]) gives p = 0.025. (verified: Run on statsmodels 0.14.6 in this repository.)
- Cohen's h thresholds of 0.2, 0.5 and 0.8 for small, medium and large. (verified: Cohen (1988), Statistical Power Analysis for the Behavioral Sciences.)

B8-L3 Many tests, and the false positives they bring

Concept

At a 5% significance level, each test of a true null hypothesis has a 5% chance of a false positive. Run 20 such tests and the chance of **at least one** false positive is `1 - 0.95**20 = 0.64`. A simulation in this course's build agreed: 20 t-tests on pure noise, repeated 2,000 times, gave at least one "significant" result 66% of the time.

Survey reports run many tests by default: every brand, every region, every age band, every wave. Some cells will light up by chance every time.

Adjust the p-values for the number of tests in the family. In statsmodels 0.14:

```
from statsmodels.stats.multitest import multipletests
reject, p_adj, _, _ = multipletests(p_values, alpha=0.05, method="holm")
```

- **Bonferroni** multiplies each p-value by the number of tests `m`. Simple and strict.

- **Holm** is a step-down version of Bonferroni. It controls the same error (any false positive in the family) and always rejects at least as many hypotheses. Prefer it to Bonferroni.
- **Benjamini-Hochberg** (`method="fdr_bh"`) controls the **false discovery rate**: the expected share of false positives among the results you flag. It is less strict and suits screening many cells for follow-up.

For p-values 0.01, 0.02, 0.03, 0.04 and 0.20, Bonferroni and Holm reject only the first. Benjamini-Hochberg rejects the first four.

Define the **family** before you look: "the 12 regional comparisons in this chart". Adjust within it. Choosing the family after seeing the results defeats the point.

Two habits that reduce the problem at the source:

- Pre-register the few comparisons that answer the business question. Test those. Label the rest as exploratory.
- Report how many tests you ran. "3 of 40 regions changed significantly" reads very differently from "Region 7 changed significantly".

Why it matters

A monthly tracker tested 14 brands in 9 regions, 126 tests, each at 5%. Every month about six cells were flagged in red on the dashboard. Regional managers were asked to explain their red cells. The following month, different cells turned red. Nobody could explain the pattern, because there was none: 126 tests at 5% produce about six false positives by chance.

Holm adjustment within each brand's family of 9 regions removed nearly all the monthly red cells. The few that remained repeated in the next month.

Worked example: weak code

```
for region, p in p_values.items():
    if p < 0.05:
        flag(region)
```

1. **Line 2.** Each test at 5%, with no adjustment for how many run: false positives are expected.

Worked example: strong code

```
from statsmodels.stats.multitest import multipletests


def flag_regions(p_values: dict[str, float], alpha: float = 0.05) ->
    dict[str, float]:
```

```
5     """Holm-adjusted p-values for one family of regional tests, flagged
      below alpha."""
6     regions = list(p_values)
1     reject, p_adj, _, _ = multipletests([p_values[r] for r in regions],
      alpha=alpha, method="holm")
2     return {r: float(p) for r, p, rej in zip(regions, p_adj, reject,
      strict=True) if rej}
```

1. **Line 7.** One family, adjusted together with Holm.
2. **Line 8.** Report adjusted p-values, so readers see the correction.

Define the family first, adjust within it with Holm or Benjamini-Hochberg, and report how many tests ran.

Recap card

- **Chance of at least one false positive in 20 tests at 5%** 1 - 0.95^20 = 0.64.
- **Holm vs Bonferroni** Same family-wise control. Holm is step-down and rejects at least as many. Prefer Holm.
- **Benjamini-Hochberg** Controls the false discovery rate. Suits screening many cells.
- **Define the family** Before looking at results. Report how many tests ran.

Practice

Exercise 1. Multiple choice (level 1)

You run 20 independent tests at the 5% level. All null hypotheses are true. What is the chance of at least one significant result?

- a) 5%
- b) About 64%
- c) 100%
- d) 0%, because the nulls are true

Why?

- r1) 1 - 0.95^20 = 0.64.
- r2) 20 × 5% = 100%, minus overlap.

Exercise 2. Predict the output (level 2)

What does this print? (statsmodels 0.14)

```
1 from statsmodels.stats.multitest import multipletests
2
3 p = [0.01, 0.02, 0.03, 0.04, 0.20]
```

```
print(multipletests(p, alpha=0.05, method="holm")[0].sum(),
    multipletests(p, alpha=0.05, method="fdr_bh")[0].sum())
```

Exercise 3. Fill the gap (level 2)

Adjust a family of p-values with Holm's method.

```
reject, p_adj, _, _ = _____(1)(p_values, alpha=0.05, method="_____(2)")
```

Exercise 4. Write the function (level 3)

Write `holm_adjust(p_values: list[float]) -> list[float]` without statsmodels. Return adjusted p-values **in the input order**.

1. Sort the p-values ascending. For the k-th smallest (k from 0), compute `min(1, (m - k) * p)`, where `m` is the number of p-values.
2. Walk the sorted list and keep a running maximum, so adjusted values never decrease.
3. Put each adjusted value back at its original position.

Your code must pass these checks:

- Multiply the k-th smallest by (m - k) and keep a running maximum.
- Return adjusted values in the input order, not sorted order.
- Cap at 1.0, and return an empty list for no input.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Holm step-down adjusted p-values, in input order.
def holm_adjust(p_values: list[float]) -> list[float]: ...
```

Starter code:

```
def holm_adjust(p_values):
    m = len(p_values)
    return [min(1.0, p * m) for p in p_values]
```

Exercise 5. Architecture design (level 4)

Choose the approach for each analysis.

Options: Holm, family of the three, Benjamini-Hochberg, No adjustment.

1. Three pre-registered comparisons that decide a campaign's budget
2. Screen 400 item-by-segment cells to pick a few for follow-up research
3. One primary hypothesis, stated before fieldwork

Sources

- multipletests on [0.01, 0.02, 0.03, 0.04, 0.20]: Holm and Bonferroni reject one, fdr_bh rejects four. (verified: Run on statsmodels 0.14.6 and 0.15.0 in this repository.)
- 20 t-tests on noise, 2,000 repetitions: at least one $p < 0.05$ in 66% of repetitions. (verified: Simulated with NumPy 2.4.6 and SciPy 1.17.1 in this repository.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B8-L1 Survey weights and the effective sample size

B8-L1 exercise 1

Hints. (1) What do unequal weights do to precision? (2) Which number measures the remaining information? (3) Kish's n_eff.

Answer. b) 71 Because: r1) Unequal weights reduce precision, and n_eff measures the information that remains.

Use `n_eff` for standard errors, intervals and tests. Report the raw base alongside.

Common wrong answers.

- `a`: Raw n overstates precision when weights vary, so intervals come out too narrow.
- `c`: The weight total depends on the scaling of the weights, not on the information in the sample.
- `d`: That difference has no statistical meaning.
- `b|r2`: Right answer, wrong reason. n_eff is the correct amount of information, not a cautious guess.

B8-L1 exercise 2

Hints. (1) Which respondents answered 1, and what are their weights? (2) Sum of those weights over the total weight. (3) 5.5 / 8.

Answer.

```
1 0.6875 0.6
```

Weighted: 5.5 / 8 = 0.6875. Unweighted: 3 / 5 = 0.6.

Common wrong answers.

- `0.6 0.6`: np.average with weights gives (0.5 + 1 + 4) / 8 = 0.6875.
- `1.1 0.6`: That is sum(y * w) / n. A weighted mean divides by sum(w) = 8.

B8-L1 exercise 3

Hints. (1) Sum of weights, squared, over the sum of squared weights. (2) Design effect = raw n / n_eff. (3) `2` and `len(w)`.

Answer.

```
n_eff = w.sum() ** 2 / (w ** 2).sum()
deff = len(w) / n_eff
```

`n_eff = (Σw)² / Σw²`. `deff = n / n_eff`. Equal weights give 1.

Common wrong answers.

- g2 = `w.sum()`: The design effect compares the raw count with n_eff: len(w) / n_eff.

B8-L1 exercise 4

Hints. (1) Drop rows missing either column first. (2) Divide by the weight total, and return None when it is 0. (3) n_eff = w.sum() ** 2 / (w ** 2).sum().

Answer.

```python
import pandas as pd


def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float | None:
    """Weighted mean of value over rows with both value and weight present.

    Returns:
        The weighted mean, or None for no rows or a zero weight total.
    """
    d = df[[value, weight]].dropna()
    total = d[weight].sum()
    if len(d) == 0 or total == 0:
        return None
    return float((d[value] * d[weight]).sum() / total)


def effective_n(weights: pd.Series) -> float | None:
    """Kish effective sample size, (sum w)^2 / sum(w^2), ignoring missing weights.

    Returns:
        n_eff, or None when there are no non-zero weights.
    """
    w = weights.dropna()
    sum_sq = (w**2).sum()
    if sum_sq == 0:
        return None
    return float(w.sum() ** 2 / sum_sq)
```

Weighted means divide by the weight total. `n_eff` tells you how much information the weighted sample holds.

B8-L1 exercise 5

Hints. (1) Which describes the population? (2) Which measures information? (3) Which tells readers how many answered?

Answer.

- Chart: the weighted share, which estimates the population.
- Interval: n_eff, so precision reflects the weights.
- Footnote: the raw base, so readers know how many people answered.
- Suppression: n_eff, so a heavily weighted small group is not reported as if it were large.

Weighted for estimates, n_eff for precision and suppression, raw n for transparency.

Common wrong answers.

- estimate = `unweighted`: The unweighted share describes the sample, not the population.
- ci = `raw_n`: Raw n overstates precision when weights vary.
- suppress = `raw_n`: A group of 60 with n_eff 25 carries the information of 25 people. Compare n_eff with the threshold.

B8-L2 Confidence intervals and effect sizes

B8-L2 exercise 1

Hints. (1) What is p(1-p) when p = 0? (2) What is the half-width then? (3) Zero.

Answer. a) [0, 0] Because: r1) With p = 0 the standard error is 0, so the interval collapses.

Wald claims certainty from 20 people. Wilson gives [0, 0.161].

Common wrong answers.

- `b`: That is the Wilson interval. Wald has zero width at p = 0.
- `c`: p(1-p) is 0, so the half-width is 0, not 0.05.
- `d`: sqrt(0) is 0. No error, just a misleading interval.
- `a|r2`: Right answer, wrong reason. Wilson gives a sensible interval for 20. Wald fails at the edge.

B8-L2 exercise 2

Hints. (1) How large is each group? (2) What happens to standard errors as n grows? (3) Tiny differences become significant.

Answer.

```
True
```

p = 0.025. The difference is 0.5 points, Cohen's h about 0.01. Significant is not the same as important.

Common wrong answers.

- `False`: With 100,000 per group, a 0.5-point difference gives p = 0.025. Significant, and still tiny.

B8-L2 exercise 3

Hints. (1) alpha is 1 minus the confidence level. (2) The method named after Edwin Wilson. (3) `0.05` and `wilson`.

Answer.

```
low, high = proportion_confint(count=40, nobs=100, alpha=0.05,
    method="wilson")
```

`alpha=0.05, method="wilson"` gives (0.309, 0.498).

Common wrong answers.

- g1 = `0.95`: alpha is the error rate. A 95% interval uses alpha=0.05.
- g2 = `normal`: normal is the Wald interval, which fails near 0 and 1.

B8-L2 exercise 4

Hints. (1) Guard first: n <= 0 or p outside [0, 1] returns None. (2) Compute denom, centre and half in that order. (3) Clip with max(0.0, ...) and min(1.0, ...).

Answer.

```python
import math


def wilson_interval(p: float, n: float, z: float = 1.959964) ->
    tuple[float, float] | None:
    """Wilson score interval for a share.

    Args:
        p: Observed share, weighted or not.
        n: Sample size, or n_eff for a weighted share.
        z: Normal quantile. 1.959964 gives 95%.

    Returns:
```

```
        (low, high) clipped to [0, 1], or None for invalid input.
    """
    if n <= 0 or not 0 <= p <= 1:
        return None
    denom = 1 + z**2 / n
    centre = (p + z**2 / (2 * n)) / denom
    half = z / denom * math.sqrt(p * (1 - p) / n + z**2 / (4 * n**2))
    return (max(0.0, centre - half), min(1.0, centre + half))
```

Wilson shifts the centre towards 0.5 and widens the interval when information is thin, so it never collapses to zero width.

B8-L2 exercise 5

Hints. (1) Is 0.04 bigger than 0.2? (2) What does a reader need to judge size? (3) Set the threshold before looking.

Answer.

- Headline: no meaningful change. 0.04 is far below the agreed 0.2.
- Detail: the difference and its interval, next to the threshold, so readers see the size.
- Rule: decide the meaningful size in advance and judge against it.

Size decides meaning. Significance only says the size is not zero.

Common wrong answers.

- headline = `sig_up`: True, and misleading. With 90,000 respondents almost any change is significant.
- detail = `p_only`: A p-value says nothing about size. Show the difference and its interval.
- rule = `p_rule`: In large samples this flags trivial changes every quarter.

B8-L3 Many tests, and the false positives they bring

B8-L3 exercise 1

Hints. (1) What is the chance one test is not significant? (2) And all 20? (3) 1 - 0.95**20.

Answer. b) About 64% Because: r1) 1 - 0.95^20 = 0.64.

The chance none of 20 fire is 0.95^20 = 0.36, so at least one fires with probability 0.64.

Common wrong answers.

- `a`: 5% is the rate for one test. The family has 20 chances.

- `c`: 20 × 5% is the expected number of false positives (1), not a probability.
- `d`: True nulls still produce p < 0.05 in 5% of tests, by definition.
- `b|r2`: Right number, shaky reasoning. The exact calculation is 1 minus the chance of no false positives.

B8-L3 exercise 2

Hints. (1) Holm: 0.01×5, 0.02×4, ... (2) BH: compare the k-th smallest p with k × 0.05 / 5. (3) BH is less strict.

Answer.

```
1 1 4
```

Holm rejects only 0.01 (adjusted 0.05). BH rejects the four p-values up to 0.04, controlling the share of false discoveries instead.

Common wrong answers.

- `4 4`: Holm multiplies the smallest p by 5, the next by 4, and so on: 0.02 × 4 = 0.08 fails.
- `1 1`: Benjamini-Hochberg compares p_(k) with k/m × alpha: 0.04 ≤ 4/5 × 0.05, so four pass.

B8-L3 exercise 3

Hints. (1) The statsmodels function in stats.multitest. (2) The method name is the author's surname. (3) `multipletests` and `holm`.

Answer.

```
reject, p_adj, _, _ = multipletests(p_values, alpha=0.05, method="holm")
```

`multipletests(..., method="holm")` returns reject flags and adjusted p-values.

Common wrong answers.

- g2 = `fdr_bh`: fdr_bh is Benjamini-Hochberg. Holm's method is "holm".

B8-L3 exercise 4

Hints. (1) `order = sorted(range(m), key=lambda i: p_values[i])` gives positions from smallest p. (2) For the k-th position, `running = max(running, min(1.0, (m - k) * p))`. (3) Store at `adjusted[i]`, the original position.

Answer.

```
def holm_adjust(p_values: list[float]) -> list[float]:
    """Holm step-down adjusted p-values, in input order.

```

```
    Args:
        p_values: Raw p-values for one family of tests.

    Returns:
        Adjusted p-values, same order as the input.
    """
    m = len(p_values)
    order = sorted(range(m), key=lambda i: p_values[i])
    adjusted = [0.0] * m
    running = 0.0
    for k, i in enumerate(order):
        running = max(running, min(1.0, (m - k) * p_values[i]))
        adjusted[i] = running
    return adjusted
```

Holm scales the smallest p by m, the next by m - 1, and forces the sequence to be non-decreasing. Keeping positions lets the caller match results to tests.

B8-L3 exercise 5

Hints. (1) Is the goal to avoid any false positive, or a high share of them? (2) How many tests are in the family? (3) One test needs no adjustment.

Answer.

- Few decisive tests: Holm, which controls any false positive in the family.
- Screening many cells: Benjamini-Hochberg, which controls the share of false leads.
- A single pre-registered test: no family, so no adjustment.

Match the error rate to the decision: family-wise for confirmation, false discovery rate for screening.

Common wrong answers.

- screen = `holm`: Holm across 400 tests is so strict it flags almost nothing. For screening, control the false discovery rate.
- confirm = `none`: Three tests driving one decision form a family. Without adjustment the error rate rises to about 14%.
- single = `holm`: One test is a family of one. Holm leaves its p-value unchanged.